

Integrate the Model

2026-04-29 · lively date Hase

YOUR ML COURSE TEACHES YOU TO TRAIN MODELS. THIS BLOCK TEACHES YOU TO SHIP ONE.

The Why

Block 3 left us with a reproducible environment on both machines. The virtual environment is ready, the dependencies are pinned, the project structure is in place. What is missing is the part of the system that does any actual work: PixelWise still cannot classify anything, because the `app/` folder is empty.

ACROSS TEN LECTURE BLOCKS WE BUILD PIXELWISE. This block is the first that produces application logic. Everything before it was foundation: a place to put code, a way to track it, a reproducible environment to run it in. Everything after it depends on the inference interface produced here.

THE GAP BETWEEN A TRAINED MODEL AND A WORKING SYSTEM is larger than most students expect. A `.pkl` file sitting in a folder is not a product. It needs to be loaded safely, called with the exact input format it was trained on, and wrapped in an interface the rest of the application can depend on. Any of these can be wrong without producing an error: the model returns a prediction, the prediction is plausible, the prediction is wrong.

THIS BLOCK BRIDGES THAT GAP. We take a pre-trained digit classifier, integrate it into the PixelWise codebase, and verify end to end that it works before any API or frontend code exists. The pipeline you build here, model file in, prediction dictionary out, is the contract every later block consumes without modification.

Hands On Experience

PIXELWISE now has a scaffold, pinned dependencies, a provisioned server, and a `.env.example` that documents its configuration contract. But the `app/` directory is empty. Before we can build an API or a frontend, we need the core capability: a trained model that can classify a handwritten digit.

SILENT FAILURE is the signature bug of machine-learning systems. Code that crashes is recoverable; you read the trace and fix the line. Code that returns a wrong answer with the same confidence as a right one goes unnoticed until somebody notices the impact downstream. Most of the engineering in this block exists to turn silent failures into loud ones at the earliest possible moment.

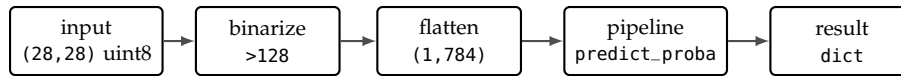


Figure 1: The inference pipeline for one image. The same pipeline scales to a batch by stacking inputs into an $(N, 28, 28)$ array; `predict_proba` is vectorised, so the cost per image stays nearly constant as N grows.

CONSIDER THIS SCENARIO: you download a model file from a colleague, drop it into your project, and write a quick script that calls `model.predict(image)`. It runs without errors. The predictions look reasonable. You ship it. Two weeks later a user reports that the app never predicts the digit 0. You check: the model was trained on digits 1 to 9 only. Nobody documented the omission. Nobody checked.

THE MODEL CONTRACT is the agreement between the training script that produced the file and the inference code that loads it. The contract has three parts: which classes the model predicts, what shape and dtype the inputs must take, and what preprocessing has already been baked in. Honouring the contract is the difference between a service that classifies digits and a service that returns plausible nonsense.

THIS FOURTH BLOCK integrates a trained model into PixelWise and builds the inference layer. By the end you will have

- understood what a model file is and why it is not source code,
- loaded the model safely and verified its class contract at startup,
- implemented a batch-first inference interface in `app/classifier.py`,
- written a smoke test that proves end-to-end correctness on real samples,
- and documented the model's capabilities and limitations in a model card.

A SILENT FAILURE is one the program never reports. Unlike a crash, it has no log line, no stack trace, no obvious moment of failure. The only way to find it is to compare the output against an expected value, and the only way to do that systematically is to write the expectation down somewhere the program can check.

THE INFERENCE INTERFACE you build today is consumed, unchanged, by Block 5's API, Block 6's database writer, Block 7's frontend, Block 8's CI pipeline, and Block 9's analytics. Five later blocks read from the function you write this afternoon. Designing it well is leverage.

What a Model File Actually Is

A `.pkl` FILE IS A BUILD ARTEFACT, not source code. You distribute it; you do not commit it. Just as a compiled binary is produced by a compiler, a model file is produced by a training script. The training script is the source of truth: it takes data, runs an algorithm, and writes weights, hyperparameters, and any preprocessing into a single serialised object.

INSPECT THE ARTEFACT in a Python shell to see what is inside:

```
import joblib
model = joblib.load("models/digit_classifier_v1.pkl")
print(type(model))
# <class 'sklearn.pipeline.Pipeline'>
print(model.classes_)
# ['1' '2' '3' '4' '5' '6' '7' '8' '9']
```

The object that comes back is a full sklearn Pipeline, not just a weights tensor. It carries the preprocessing, the classifier, and the class labels in one package, so loading the file is enough to start predicting.

LOADING A `.pkl` FROM AN UNTRUSTED SOURCE CAN EXECUTE ARBITRARY CODE. pickle and joblib deserialise objects by reconstructing them, including any code embedded in the file. This is convenient when you produced the file yourself; it is dangerous when the file came from somewhere you do not control. Treat `.pkl` files the way you treat executables: only run what you trust.

THE COURSE PROVIDES `models/digit_classifier_v1.pkl`, a LogisticRegression pipeline trained on MNIST digits 1 to 9 (9 classes, ~54,000 training samples, public domain). Class 0 is intentionally held back; students can add it later without collecting any new data.

THE TRAINING SCRIPT `train.py` is provided as supplementary material. Students who want to understand how the artefact was produced can run it: MNIST downloads automatically via `sklearn.datasets.fetch_openml`, and training takes ~30 seconds on CPU. Further reading on training practices: Karpathy, A Recipe for Training Neural Networks: <https://karpathy.github.io/2019/04/25/recipe/> Google, Rules of Machine Learning: <https://developers.google.com/machine-learning/guides/rules-of-ml>

SAFER FORMATS EXIST. ONNX stores only weights and graph structure, no executable code. SafeTensors does the same for PyTorch tensors. For this course joblib is sufficient because we control the training pipeline end to end; for any model received from outside, prefer the safer formats. ONNX: <https://onnx.ai/> SafeTensors: <https://huggingface.co/docs/safetensors>

Exercise: Inspecting the Model Artefact

CATCHING UP VIA THE REFERENCE TAG. The course maintains a public reference repository at <https://github.com/schutera/pixelwise> with a tag at the end of every block. v0.2 marks the end of Block 3; v0.3 will mark the end of this block. If you joined late or your repository has drifted, reset to the previous tag before starting:

```
git fetch origin
git reset --hard v0.2
```

This rewrites your working tree to match the end of Block 3 exactly. The state on disk is now identical to the published reference, including `setup-server.sh`, `requirements.txt`, and `.env.example`.

TREAT THE MODEL ARTEFACT AS SOMETHING YOU LOAD, not something you commit. Copy `digit_classifier_v1.pkl` (provided) into `models/`. Confirm that `.gitignore` excludes `*.pkl` so the artefact stays out of version control:

```
ls models/
git status
```

INSPECT THE ARTEFACT in a Python shell:

```
source .venv/bin/activate
python3
>>> import joblib
>>> model = joblib.load("models/digit_classifier_v1.pkl")
>>> print(type(model))
>>> print(model.classes_)
```

What type is it? What classes does it predict? Does class 0 appear? The answer to the third question determines what CLASSES should contain in the next exercise.

EXERCISES sit directly under the concept they practise: read a section, then carry it out. If your repository has drifted from the end of Block 3, the catch-up below brings it back to the known good state.

IF THE FILE APPEARS IN `git status`, your `.gitignore` from Block 3 needs an update. Add `models/*.pkl` on its own line and rerun `git status`. A model file accidentally committed once is in the history forever, even if the next commit deletes it.

Honouring the Model Contract

EVERY MODEL WAS TRAINED ON INPUTS IN A SPECIFIC FORMAT. The format is part of the contract: a 28-by-28 greyscale image, pixel values in $[0, 1]$, flattened to 784 features. Violate the format and the model predicts silently wrong: no error, no warning, just a different number than the right one. This bug class is called train/serve skew and it is the most common production ML defect.

THE MNIST MODEL EXPECTS:

- a 28×28 greyscale image,
- pixel values normalised to $[0, 1]$,
- flattened to a 784-element vector.

THE CANVAS IN THE BROWSER PRODUCES UINT8 VALUES in the range $[0, 255]$. Converting to binary (0 or 1) reduces payload size, removes anti-aliasing noise, and matches the training-time preprocessing exactly:

```
import numpy as np
raw = np.array(...)          # (28, 28) uint8
binary = (raw > 128).astype(float) # 0s and 1s
flat = binary.reshape(1, -1)  # (1, 784)
```

THE SKLEARN PIPELINE bundles preprocessing and model into one serialised object. Loading the pipeline is loading the preprocessing. There is no separate normalisation step that a future maintainer might forget; the same object that predicts also normalises, in the order the training script established.

WHY SILENT? The model takes a vector of floats and returns a class index. At no point does it know whether the floats came from the training distribution or from some unrelated source. A photograph, a random scribble, or a correctly drawn digit with the wrong dynamic range all produce the same kind of output: a class label and a confidence score. The label might be wrong, but the system has no way to tell.

BINARIZATION is aggressive but effective for MNIST. Anti-aliased grey edges become sharp black and white. The model was trained on binarized inputs, so serving raw greyscale would introduce train/serve skew immediately, with no exception to mark the moment it happened.

scikit-learn: <https://scikit-learn.org/>
 joblib: <https://joblib.readthedocs.io/>
 NumPy: <https://numpy.org/>

Exercise: Preprocessing and Binarization

PRACTISE THE PREPROCESSING STEP that matches the training contract. Load a sample MNIST digit and compare the raw and binarized representations side by side:

```
from sklearn.datasets import fetch_openml
import numpy as np

X, y = fetch_openml("mnist_784", version=1,
                    return_X_y=True, as_frame=False)
sample = X[0].reshape(28, 28).astype(np.uint8)
binary = (sample > 128).astype(float)
print(sample[:5, :5])    # raw values
print(binary[:5, :5])    # 0s and 1s
```

PLOT IF YOU HAVE A NOTEBOOK OPEN. A side-by-side image of the raw and binarized digit makes the effect visible at a glance. `matplotlib` is not in `requirements.txt` yet; install it locally with `pip install matplotlib` for this exercise only, then leave it out of the pinned requirements until a later block needs it.

NOTICE THE STRUCTURE SURVIVES.

The raw image has anti-aliased grey edges; the binarized image is sharp black and white. The digit is still recognisable, the noise is gone, and the input now matches what the model saw during training. Serving raw greyscale would mean predicting against a distribution the model has never seen.

WHY THRESHOLD AT 128? 128 is the midpoint of the `uint8` range, the simplest split between “ink” and “paper”. Otsu’s method picks a smarter threshold per image, and for noisier inputs that is sometimes worth doing. For MNIST the simple split is enough and matches the training pipeline exactly.

Designing the Inference Interface

BEFORE WRITING ANY CODE, design the contract. Start with the class roster, then the function signatures, then the loading strategy. Each is a decision that propagates: every later block reads from this interface, and every change to it ripples through the API, the database schema, and the frontend.

A NAMED CONSTANT makes the valid output set explicit and independent of the model file:

```
CLASSES = ["1", "2", "3", "4", "5", "6", "7", "8", "9"]
# must match model.classes_, verified at startup
# note: "0" is intentionally absent in v1
```

This list belongs in the code, not only in the model. The frontend, the database schema, and the API documentation all reference it. If the model is swapped for one trained on different classes, the assertion described below fires loudly at startup rather than returning silently wrong labels for the rest of the service's lifetime.

THE FUNCTION SIGNATURES fall out of the contract:

```
def classify_batch(images: np.ndarray) -> list[dict]:
    """
    Args:
        images: (N, 28, 28) uint8, values 0-255
    Returns:
        List of N dicts:
        {"prediction": str, "confidence": float,
         "scores": dict[str, float]}
    """
    if images.ndim != 3 or images.shape[1:] != (28, 28):
        raise ValueError(
            f"Expected (N,28,28), got {images.shape}")
    arr = (images > 128).astype(float).reshape(
        len(images), -1)
    probs = _pipeline.predict_proba(arr)
    return [
        {"prediction": CLASSES[p.argmax()],
         "confidence": float(p.max()),
         "scores": dict(zip(CLASSES, p.tolist()))}
        for p in probs
    ]
```

BATCH-FIRST DESIGN. What happens when 50 students submit a drawing in the same second? If the server processes each request individually, the model is called 50 times with a (1,784) array. If it batches them, the model is called once with a (50,784) array. sklearn's `predict_proba` is natively vectorised, so the cost is nearly identical. Design around batch; single-user is a special case.

```
def classify(image: np.ndarray) -> dict:
    """Single-image convenience wrapper."""
    return classify_batch(image[np.newaxis])[0]
```

LOAD _pipeline ONCE AT MODULE LEVEL:

```
import joblib, os
_pipeline = joblib.load(os.getenv("MODEL_PATH"))
assert list(_pipeline.classes_) == CLASSES, \
    f"Model/CLASSES mismatch: {_pipeline.classes_}"
```

Deserialising on every request kills throughput; the model stays in memory for the lifetime of the process. The assertion is the key teaching moment: it fires at startup if the model's classes do not match the code constant. A swapped model with the wrong classes is caught the moment the service starts, not the next time a user gets a wrong digit.

THE WRAPPER IS FOR TESTS ONLY.

The API in Block 5 always calls `classify_batch`, even for a single drawing: it passes a (1,28,28) array. When a request queue is added later to amortise inference cost, no signature changes are needed; the queue assembles a batch and the function consumes it.

MODEL_PATH FROM .env. The path is configuration, not code. Swapping the model in production becomes an env-var change plus a service restart, not a code change and a deploy. This is the same pattern as the database URL in Block 6 and the API key from Block 3.

Exercise: The Classifier Module

IMPLEMENT THE INFERENCE INTERFACE IN `app/classifier.py` with the class constant, module-level loading, the startup assertion, and the two functions:

```
import os
import joblib
import numpy as np
from dotenv import load_dotenv

load_dotenv()

CLASSES = ["1", "2", "3", "4", "5", "6", "7", "8", "9"]

_pipeline = joblib.load(os.getenv("MODEL_PATH"))
assert list(_pipeline.classes_) == CLASSES, \
    f"Model/CLASSES mismatch: {_pipeline.classes_}"

def classify_batch(images: np.ndarray) -> list[dict]:
    if images.ndim != 3 or images.shape[1:] != (28, 28):
        raise ValueError(
            f"Expected (N,28,28), got {images.shape}")
    arr = (images > 128).astype(float).reshape(
        len(images), -1)
    probs = _pipeline.predict_proba(arr)
    return [
        {"prediction": CLASSES[p.argmax()],
         "confidence": float(p.max()),
         "scores": dict(zip(CLASSES, p.tolist()))}
        for p in probs
    ]

def classify(image: np.ndarray) -> dict:
    return classify_batch(image[np.newaxis])[0]
```

UPDATE `.env` AND `.env.example` so the loader knows where to find the artefact:

```
MODEL_PATH=models/digit_classifier_v1.pkl
```

WRITE `predict.py` AS A SMOKE TEST. Load five MNIST test samples, call `classify_batch`, print the predictions next to the ground truth:

THE ASSERTION IS THE HEADLINE. Comment it out, save, restart Python, and import the module. Now mutate `CLASSES` to add a fake "10" entry, restore the assertion, restart, and observe the assertion firing. The startup error message is exactly what you want to see when a model swap goes wrong.

```
from app.classifier import classify_batch
from sklearn.datasets import fetch_openml
import numpy as np

X, y = fetch_openml("mnist_784", version=1,
                    return_X_y=True, as_frame=False)
images = X[:5].reshape(-1, 28, 28).astype(np.uint8)
truth = y[:5]
results = classify_batch(images)
for r, t in zip(results, truth):
    print(f"Pred: {r['prediction']} "
          f"(conf {r['confidence']:.2f}) True: {t}")
```

READ `predict.py` AS THE API
PREVIEW. This is what the API in
Block 5 will do: take pixel data, call
`classify_batch`, return the result.
Block 5 swaps `stdin` for HTTP and
`stdout` for JSON; the inference logic is
unchanged.

The Model Card

A MODEL CARD is the documentation artefact that links a model to its provenance and limitations. It answers four questions any consumer of the model should know the answer to before they trust it: where did the data come from, what can the model do, what does it fail at, and what is it intended for.

FOR PIXELWISE the model card lives next to the artefact in `models/MODELCARD.md` and is committed to Git, so it travels with the repository even though the `.pkl` itself does not:

```
# MODELCARD.md: digit_classifier_v1

## Training Data
MNIST digits 1-9, ~54k train / ~9k test, public domain.
Class 0 withheld intentionally.

## Capabilities
Predict handwritten digits 1-9.
Expected accuracy: ~92% (LogisticRegression baseline).

## Known Failures
6/9 confusion, 3/8 confusion, messy or rotated digits.

## Intended Use
28x28 canvas drawings.
Out of scope: photos, non-digit characters.
```

WHAT THE CARD IS NOT is a substitute for the runtime checks already built into the loader. Documentation tells humans what to expect; the assertion at module load tells the program what to expect. Both are needed. Documentation drifts; assertions cannot.

Model cards as a concept were proposed by Mitchell et al. in 2018 and are now standard at Google, HuggingFace, and increasingly across the industry.

Model Cards (Google):
<https://modelcards.withgoogle.com/>
 HuggingFace Model Cards:
<https://huggingface.co/docs/hub/model-cards>

THE CARD IS A CONTRACT, like any other. When v2 is deployed via the Block 8 feature flag, the card is updated alongside it; the version number, the training data section, and the capabilities all change in lockstep. Traceability across versions is the entire reason the card is a separate file.

Exercise: The Model Card

WRITE THE DOCUMENTATION ARTEFACT that travels with the project even though the `.pkl` does not. Write `models/MODELCARD.md` following the structure from the theory section: dataset, input format, performance per class, known failures, version, and date.

COMMIT THE NEW FILES, but not the `.pkl`:

```
git add app/classifier.py predict.py \
    models/MODELCARD.md .env.example
git commit -m "Add classifier module and model card"
git push
```

The repository is now at the state `v0.3` marks: a working classifier, a smoke test, a model card, and an updated configuration contract. The `.pkl` stays on each developer's machine, the artefact path is in `.env`, and the rest of the team can reproduce the environment from the code in Git.

USE THE VALUES YOU MEASURED.
Replace "`~92%`" with whatever `predict.py` reports as you expand it across the test set. The numbers you cite should match the numbers your code produces; a model card with optimistic round figures is the start of the documentation drift the previous section taught us to avoid.

VERIFY THE SMOKE TEST ON prod.
SSH into the server, pull, copy the `.pkl` into `/opt/pixelwise/models/`, set `MODEL_PATH` in the production `.env`, and run `python predict.py`. Identical output on both machines is the deliverable of this block.

Optional: Add the Missing o

THE DIGIT CLASS "0" WAS INTENTIONALLY EXCLUDED FROM v1. To produce v2: add "0" to CLASSES, include MNIST's class-0 samples in the training split, re-run `train.py`, save as `digit_classifier_v2.pkl`. The assertion will break the moment v2 is loaded against v1's CLASSES; fix it by updating the constant. Update `MODEL_CARD.md` to document that v2 covers all ten digits and note any accuracy change.

COULD WE HAVE SPOTTED THE MISSING CLASS AT RUNTIME?

- After a few hundred predictions, a missing class is invisible: the model never predicts it, users never see an error, the database just never accumulates any os.
- When the model sees an input it was not trained on, it is often less certain. A systematic drop in average confidence could indicate out-of-distribution inputs: a weak signal, but a runtime signal that requires no labelled data.
- Returning `null` when `confidence < threshold` turns low confidence into a monitoring signal: the volume of rejected predictions is itself a metric.

A NOTE ON WHAT COMES NEXT. `app/classifier.py` works, but it is trapped in a Python script. A user with a browser cannot call it; a teammate on a different machine cannot reach it. The next block wraps `classify_batch` in an HTTP endpoint, defines the request and response contract with Pydantic, and runs the result as a managed systemd service so the model becomes reachable without a human in the loop.

This challenge connects directly to Block 8: v2 is deployed via the feature flag, and Block 9's analytics compare v1 and v2 performance. No data collection is needed; MNIST has the os already.

SYSTEMATIC DETECTION OF SILENT FAILURES is the domain of MLOps: uncertainty estimation, distribution shift, missing classes, silent degradation. This surface is intentionally shallow here. A dedicated MLOps course covers it in depth.

Self-Reflection and Recap

SELF-REFLECTION questions to guide your thoughts during and after the exercises:

- What is train/serve skew, and why is it the most common production ML defect?
- Why do we load the model once at module level instead of on every request?
- What does the startup assertion protect against, and what would happen without it?
- Why is the `.pkl` in `.gitignore` but `MODEL_CARD.md` is committed?
- How would you detect at runtime that the model is missing a class it should predict?
- Why is batch-first design the right default, even when most requests carry a single image?

RECAP of key concepts:

- A `.pkl` file is a build artefact, not source code; never commit it to Git.
- Train/serve skew is silent by default; the startup assertion is the loudest defence.
- Batch-first design scales from one user to N concurrent users with no code changes.
- Module-level loading keeps the model in memory for the lifetime of the process.
- The model card documents capabilities, limitations, and provenance, and updates with the model.
- `predict.py` proves end-to-end correctness before any API exists.

MILESTONE. `app/classifier.py` exposes a batch-first inference interface that scales from one student to N concurrent users with no code changes. `predict.py` proves it works end to end on real MNIST samples. `MODEL_CARD.md` documents what the model is and is not. The next chapter wraps the inference function in a FastAPI service so the model becomes reachable over HTTP.

SECURITY THREAD. Never load a `.pkl` from an untrusted source; `joblib` and `pickle` deserialisation execute arbitrary code. `MODEL_PATH` comes from `.env`, never hard-coded, so swapping the model in production requires only an env-var change, not a code change. Training data and model artefacts are not committed to Git.

TEASER. The classifier is callable from Python, but who, outside your terminal, can reach it?